Case Study on a wellness company using Python

Connect The Data:

1. Importing the Required Tools In Jupyter Notebook

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('ggplot')
import datetime as dt
#pd.set_option('max_column')
```

input tools

2. Importing the Data for the Case Study

In [3]:
```
#Prepare the Data
df = pd.read_csv('/kaggle/input/datasets/arashnic/fitbit/mturkfitbit_export_4.12.16-5.12.16/Fitabase Data 4.12.16-5.12.16/dailyActivity_merged.csv')
```

files path from where it is used

3. To Check the Total Rows and Columns Present in the Data

In [4]:
```
df.shape
```

Out[4]:
```
(940, 15)
```

there is 940 Rows and 15 columns from DailyActivity_merged Data

4. To Check the Column Names

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Id', 'ActivityDate', 'TotalSteps', 'TotalDistance', 'TrackerDistance',
       'LoggedActivitiesDistance', 'VeryActiveDistance',
       'ModeratelyActiveDistance', 'LightActiveDistance',
       'SedentaryActiveDistance', 'VeryActiveMinutes', 'FairlyActiveMinutes',
       'LightlyActiveMinutes', 'SedentaryMinutes', 'Calories'],
      dtype='object')
```

columns it is include names

5. Select the Top 10 Rows and Columns From the Data

In [6]:
```
df.head(10)
```

Out[6]:

	Id	ActivityDate	TotalSteps	TotalDistance	TrackerDistance	LoggedActivitiesDistance	VeryActiveDistance	ModeratelyActive
0	1503960366	4/12/2016	13162	8.50	8.50	0.0	1.88	0.55
1	1503960366	4/13/2016	10735	6.97	6.97	0.0	1.57	0.69
2	1503960366	4/14/2016	10460	6.74	6.74	0.0	2.44	0.40
3	1503960366	4/15/2016	9762	6.28	6.28	0.0	2.14	1.26
4	1503960366	4/16/2016	12669	8.16	8.16	0.0	2.71	0.41
5	1503960366	4/17/2016	9705	6.48	6.48	0.0	3.19	0.78
6	1503960366	4/18/2016	13019	8.59	8.59	0.0	3.25	0.64
7	1503960366	4/19/2016	15506	9.88	9.88	0.0	3.53	1.32
8	1503960366	4/20/2016	10544	6.68	6.68	0.0	1.96	0.48
9	1503960366	4/21/2016	9819	6.34	6.34	0.0	1.34	0.35

selecting the top 10 rows and columns

6. Check the DataTypes of Columns

In [7]:
```
df.dtypes
```

Out[7]:
```
Id                          int64
ActivityDate               object
TotalSteps                  int64
TotalDistance             float64
TrackerDistance           float64
LoggedActivitiesDistance  float64
VeryActiveDistance        float64
ModeratelyActiveDistance  float64
LightActiveDistance       float64
SedentaryActiveDistance   float64
VeryActiveMinutes           int64
FairlyActiveMinutes         int64
LightlyActiveMinutes        int64
SedentaryMinutes            int64
Calories                    int64
dtype: object
```

data types or data store as

Clean The Data

7. Change the Date Format To M-D-Y Format

In [8]:
```
#Clean The Data
df['Id'] = df['Id'].astype(str)
df['ActivityDate'] = pd.to_datetime(df['ActivityDate'], format= '%m/%d/%Y')
df.dtypes
```

Out[8]:
```
Id                                object
ActivityDate              datetime64[ns]
TotalSteps                         int64
TotalDistance                    float64
TrackerDistance                  float64
LoggedActivitiesDistance         float64
VeryActiveDistance               float64
ModeratelyActiveDistance         float64
LightActiveDistance              float64
SedentaryActiveDistance          float64
VeryActiveMinutes                  int64
FairlyActiveMinutes                int64
LightlyActiveMinutes               int64
SedentaryMinutes                   int64
Calories                           int64
dtype: object
```

change the id datetype from **Integer to String** and ActivityDate datatype from **Object to datetime**

8. Creating a new Column To Find Out The Distance Difference

In [9]:
```
df['distance_diff'] = df['TotalDistance'] - df['TrackerDistance']
```

giving a comment to check the **difference between TotalDistance and TrackerDistance** whether it is same or Difference

9. Finding the Difference Between the Two Columns

In [10]:
```
df['distance_diff'].value_counts()
```

Out[10]:

distance_diff	
0.000000	925
1.830000	1
0.190001	1
0.040000	1
0.810000	1
1.049999	1
0.760000	1
1.070000	1
0.980000	1
0.900001	1
1.140000	1
1.160000	1
0.880000	1
0.460000	1
1.160000	1
1.060000	1

Name: count, dtype: int64

find the Difference between the two

10. Finding out any Duplicated Values Present or Not

In [11]:
```
df.query('distance_diff > 0.0')
```

Out[11]:

	Id	ActivityDate	TotalSteps	TotalDistance	TrackerDistance	LoggedActivitiesDistance	VeryActiveDistance	ModeratelyAc
689	6962181067	2016-04-21	11835	9.71	7.88	4.081692	3.99	2.10
693	6962181067	2016-04-25	13239	9.27	9.08	2.785175	3.02	1.68
707	6962181067	2016-05-09	12342	8.72	8.68	3.167822	3.90	1.18
711	7007744171	2016-04-12	14172	10.29	9.48	4.869783	4.50	0.38
712	7007744171	2016-04-13	12862	9.65	8.60	4.851307	4.61	0.56
713	7007744171	2016-04-14	11179	8.24	7.48	3.285415	2.95	0.34
717	7007744171	2016-04-18	14816	10.98	9.91	4.930550	3.79	2.12
718	7007744171	2016-04-19	14194	10.48	9.50	4.942142	4.41	0.76
719	7007744171	2016-04-20	15566	11.31	10.41	4.924841	4.79	0.67
724	7007744171	2016-04-25	18229	13.34	12.20	4.861792	4.31	1.37
726	7007744171	2016-04-27	13541	10.22	9.06	4.685605	4.27	0.66
728	7007744171	2016-04-29	20067	14.30	13.42	4.911146	4.31	2.05
731	7007744171	2016-05-02	13041	9.18	8.72	2.832326	4.64	0.70
732	7007744171	2016-05-03	14510	10.87	9.71	4.912368	4.48	1.02
734	7007744171	2016-05-05	15010	11.10	10.04	4.878232	4.33	1.29

check using query if any as the value difference from each other, there are some value that are not equal

11. Changing the Column Names into Lowercase

In [12]:
```
df.columns = df.columns.str.lower()
df.columns
```

Out[12]:
```
Index(['id', 'activitydate', 'totalsteps', 'totaldistance', 'trackerdistance',
       'loggedactivitiesdistance', 'veryactivedistance',
       'moderatelyactivedistance', 'lightactivedistance',
       'sedentaryactivedistance', 'veryactiveminutes', 'fairlyactiveminutes',
       'lightlyactiveminutes', 'sedentaryminutes', 'calories',
       'distance_diff'],
      dtype='object')
```

change the columns name into lower case

12. Rename the Column Names

In [13]:
```
df.rename(columns = {'activitydate':'activity_date','totalsteps':'total_steps','totaldistance':'total_di
stance','trackerdistance':'tracker_distance',
       'loggedactivitiesdistance' : 'logged_activities_distance', 'veryactivedistance' : 'very_active_di
stance',
       'moderatelyactivedistance' : 'moderately_active_distance', 'lightactivedistance': 'light_active_d
istance',
       'sedentaryactivedistance' : 'sedentary_active_distance', 'veryactiveminutes' : 'very_active_minut
es', 'fairlyactiveminutes': 'fairly_active_minutes',
       'lightlyactiveminutes' : 'lightly_active_minutes', 'sedentaryminutes' : 'sedentary_minutes'},inpl
ace = True)

df.columns
```

Out[13]:
```
Index(['id', 'activity_date', 'total_steps', 'total_distance',
       'tracker_distance', 'logged_activities_distance',
       'very_active_distance', 'moderately_active_distance',
       'light_active_distance', 'sedentary_active_distance',
       'very_active_minutes', 'fairly_active_minutes',
       'lightly_active_minutes', 'sedentary_minutes', 'calories',
       'distance_diff'],
      dtype='object')
```

change the columns name by rename and to check the columns. **IF YOU WANT TO CHECK THE DATA BY ENTER THE INPLACE AS FALSE** . we can the data if it is true it will be saved

13. Change the Date Column into Day of Week by creating a new Column

In [14]:
```
#Create Columns.
day_of_week = df['activity_date'].dt.day_name()
df['day_of_week'] = day_of_week


df['n_day_of_week'] = df['activity_date'].dt.weekday #0 monday 6 sunday
```

create column for the day of week by numbering the week from 0 to 6 sarting from monday to sunday

14. Re-Checking the Column Names

In [16]:
```
df.columns
```

Out[16]:
```
Index(['id', 'activity_date', 'total_steps', 'total_distance',
       'tracker_distance', 'logged_activities_distance',
       'very_active_distance', 'moderately_active_distance',
       'light_active_distance', 'sedentary_active_distance',
       'very_active_minutes', 'fairly_active_minutes',
       'lightly_active_minutes', 'sedentary_minutes', 'calories',
       'distance_diff', 'day_of_week', 'n_day_of_week'],
      dtype='object')
```

check the day_of_week

15. Check for the Null Values in Every Column (there are 2 options to check Null Values)

In [17]:
```
#Checking the Null_values
df.isna().sum()
```

Out[17]:

id	0
activity_date	0
total_steps	0
total_distance	0
tracker_distance	0
logged_activities_distance	0
very_active_distance	0
moderately_active_distance	0
light_active_distance	0
sedentary_active_distance	0
very_active_minutes	0
fairly_active_minutes	0
lightly_active_minutes	0
sedentary_minutes	0
calories	0
distance_diff	0
day_of_week	0
n_day_of_week	0

dtype: int64

check the null values by isna code and using the sum code we can see the columns wise null values

In [18]:
```
#other code to check the Null Value
df.isnull().sum()
```

Out[18]:

id	0
activity_date	0
total_steps	0
total_distance	0
tracker_distance	0
logged_activities_distance	0
very_active_distance	0
moderately_active_distance	0
light_active_distance	0
sedentary_active_distance	0
very_active_minutes	0
fairly_active_minutes	0
lightly_active_minutes	0
sedentary_minutes	0
calories	0
distance_diff	0
day_of_week	0
n_day_of_week	0

dtype: int64

16. Checking the Duplicated Values in Every Column

In [19]:
```
#checking the duplicates
df.duplicated().sum()
```

Out[19]:
```
np.int64(0)
```

check for the duplicates

Modelling The Data

17. Select the Required Data

In [20]:
```
#subset the data
df = df[['id', 'activity_date', 'total_steps', 'total_distance',
       'very_active_minutes', 'fairly_active_minutes',
       'lightly_active_minutes', 'sedentary_minutes', 'calories',
       #'activity_level',
         'day_of_week', 'n_day_of_week']].copy()
```

select the only used column and unselect the unused data and make a copy of the data

18. Checking the Data

In [21]:
```
df.head(3)
```

Out[21]:

	id	activity_date	total_steps	total_distance	very_active_minutes	fairly_active_minutes	lightly_active_minutes	sedentary_mir
0	1503960366	2016-04-12	13162	8.50	25	13	328	728
1	1503960366	2016-04-13	10735	6.97	21	19	217	776
2	1503960366	2016-04-14	10460	6.74	30	11	181	1218

checking the copyed data

Analyzing The Data

19. GroupBy using Categories

In [22]:

```
# Analysis
 # 1.Categories
      # sedentary: Less than 6000 on Average
      # Active : between 6000 and 12000 on Average
     # very active: More than 12000 on Average

# to find the total sum values
     #id_grp = df.groupby(['id'])
               # or
     #df['activity_date'].value_counts()

id_grp = df.groupby(['id'])
id_avg_step = id_grp['total_steps'].mean().sort_values(ascending=False)
id_avg_step = id_avg_step.to_frame()

conditions = [
      (id_avg_step <= 6000),
      (id_avg_step > 6000) & (id_avg_step < 12000),
      (id_avg_step >= 12000)
    ]
values = ['sedentary', 'active', 'very active']

id_avg_step['activity_level'] = np.select(conditions,values, default='')

id_activity_level = id_avg_step['activity_level']

id_avg_step
```

id	total_steps	activity_level
8877689391	16040.032258	very active
8053475328	14763.290323	very active
1503960366	12116.741935	very active
2022484408	11370.645161	active
7007744171	11323.423077	active
3977333714	10984.566667	active
4388161847	10813.935484	active
6962181067	9794.806452	active
2347167796	9519.666667	active
7086361926	9371.774194	active
8378563200	8717.709677	active
5553957443	8612.580645	active
4702921684	8572.064516	active
5577150313	8304.433333	active
4558609924	7685.129032	active
2873212765	7555.774194	active
1644430081	7282.966667	active
4319703577	7268.838710	active
8583815059	7198.516129	active
6117666160	7046.714286	active
3372868164	6861.650000	active
8253242879	6482.157895	active
1624580081	5743.903226	sedentary
6290855005	5649.551724	sedentary
2026352035	5566.870968	sedentary
4445114986	4796.548387	sedentary
2320127002	4716.870968	sedentary
4057192912	3838.000000	sedentary
1844505072	2580.064516	sedentary
6775888955	2519.692308	sedentary
4020332650	2267.225806	sedentary
8792009665	1853.724138	sedentary
1927972279	916.129032	sedentary

give a condition to check which or

- less or equal to 6000
- more than 6000 or less than 12000
- more than 12000

for this code using default will be useful otherwise it will be a error

20. To Add the Categories in my Current Copy

In [23]:
```
df['activity_level'] = [id_activity_level[c] for c in df['id']]

df.head(4)
```

Out[23]:

	id	activity_date	total_steps	total_distance	very_active_minutes	fairly_active_minutes	lightly_active_minutes	sedentary_mir
0	1503960366	2016-04-12	13162	8.50	25	13	328	728
1	1503960366	2016-04-13	10735	6.97	21	19	217	776
2	1503960366	2016-04-14	10460	6.74	30	11	181	1218
3	1503960366	2016-04-15	9762	6.28	29	34	209	726

to add the data to the current copy one

21. Checking the ID Column Datatype

In [24]:
```
df['id'].value_counts()
```

Out[24]:
```
id
1503960366    31
1624580081    31
1844505072    31
1927972279    31
2022484408    31
2320127002    31
2026352035    31
4020332650    31
2873212765    31
4445114986    31
4319703577    31
4388161847    31
8378563200    31
6962181067    31
5553957443    31
4702921684    31
4558609924    31
8877689391    31
8583815059    31
8053475328    31
7086361926    31
1644430081    30
5577150313    30
3977333714    30
6290855005    29
8792009665    29
6117666160    28
7007744171    26
6775888955    26
3372868164    20
8253242879    19
2347167796    18
4057192912     4
Name: count, dtype: int64
```

22. Checking the Data Finally

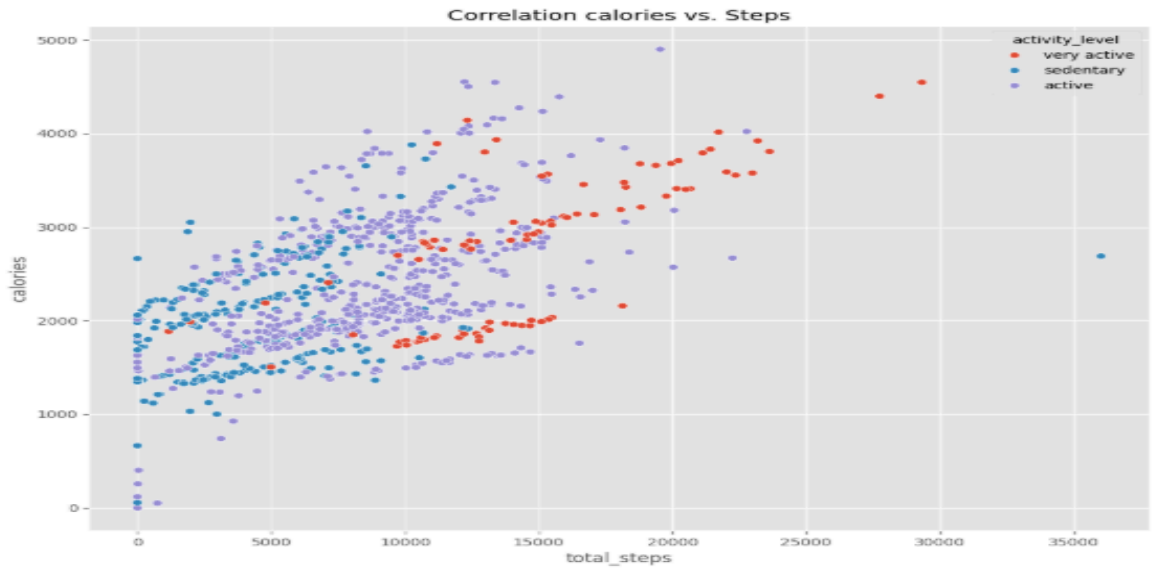
In [25]:
```
df.describe()
```

Out[25]:

	activity_date	total_steps	total_distance	very_active_minutes	fairly_active_minutes	lightly_active_minutes	sedentary_m
count	940	940.000000	940.000000	940.000000	940.000000	940.000000	940.00000
mean	2016-04-26 06:53:37.021276672	7637.910638	5.489702	21.164894	13.564894	192.812766	991.21063
min	2016-04-12 00:00:00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	2016-04-19 00:00:00	3789.750000	2.620000	0.000000	0.000000	127.000000	729.75000
50%	2016-04-26 00:00:00	7405.500000	5.245000	4.000000	6.000000	199.000000	1057.5000
75%	2016-05-04 00:00:00	10727.000000	7.712500	32.000000	19.000000	264.000000	1229.5000
max	2016-05-12 00:00:00	36019.000000	28.030001	210.000000	143.000000	518.000000	1440.0000
std	NaN	5087.150742	3.924606	32.844803	19.987404	109.174700	301.26743

23. Correlation between Calories and Steps

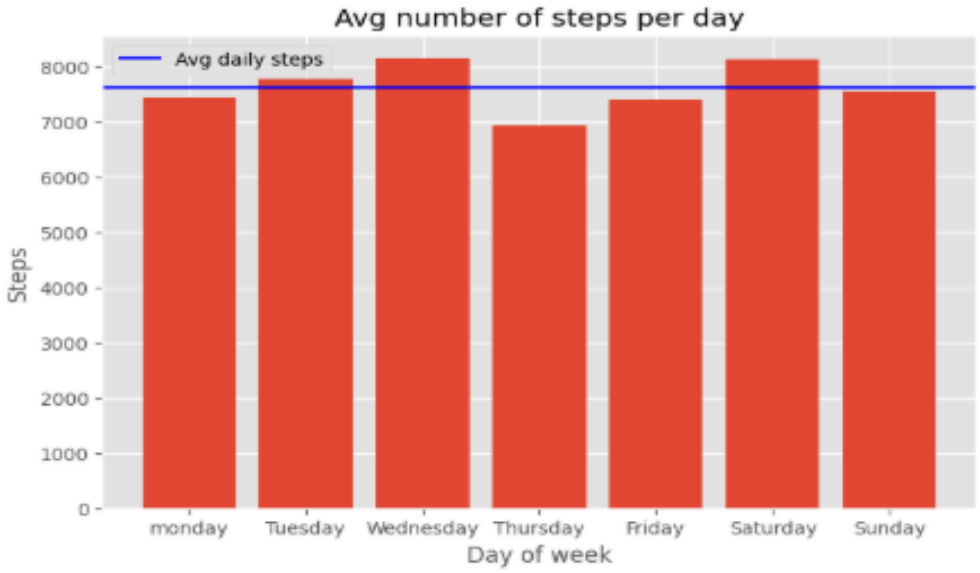
In [26]:
```
#Share
# Correlation Between Steps and Calories Burned

plt.figure(figsize=(10,8))
ax = sns.scatterplot(x='total_steps',y ='calories',data= df, hue= df['activity_level'])
plt.title('Correlation calories vs. Steps')

plt.tight_layout()
plt.show()
```



24. Average Number of Steps By Week

In [27]:

```
#Average Steps per Day

day_of_week = ['monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
fig, ax = plt.subplots(1,1,figsize = (8,5))

day_grp = df.groupby(['day_of_week'])
avg_daily_steps = day_grp['total_steps'].mean()
avg_steps = df['total_steps'].mean()

plt.bar(avg_daily_steps.index, avg_daily_steps)

ax.set_xticks(range(len(day_of_week)))
ax.set_xticklabels(day_of_week)

ax.axhline(y = avg_daily_steps.mean() ,color= 'blue' ,label='Avg daily steps')

ax.set_ylabel('Steps')
ax.set_xlabel('Day of week')
ax.set_title('Avg number of steps per day')
plt.legend()
plt.show()
```



25. Percentage of activity in minutes

In [28]:

```
#percentage of activity in Minutes

very_active_mins = df['very_active_minutes'].sum()
fairly_active_mins = df['fairly_active_minutes'].sum()
lightly_active_mins = df['lightly_active_minutes'].sum()
sedentary_mins = df['sedentary_minutes'].sum()

slices = [very_active_mins,fairly_active_mins,lightly_active_mins,sedentary_mins]

labels = ['very_active_mins', 'fairly_active_mins', 'lightly_active_mins', 'sedentary_mins']

explode = [0,0,0,0.1]

plt.pie(slices, labels= labels, explode = explode, autopct = '%1.1f%%')

plt.title('% of activity level in minutes')
plt.show()
```

26. Correlation Between Activity level in minutes and Calories

```
# Correlation Between Activity level in minutes and Calories

n_day_of_week = [0,1,2,3,4,5,6]

fig, axes = plt.subplots(nrows = 2,ncols= 2, figsize= (8,9) ,dpi=70)

sns.scatterplot(data=df, x = 'calories', y ='sedentary_minutes', hue='activity_level', ax = axes[0,0],leg
end= False)
sns.scatterplot(data=df, x = 'calories', y ='lightly_active_minutes', hue='activity_level', ax = axes[0,
1],legend= False)
sns.scatterplot(data=df, x = 'calories', y ='fairly_active_minutes', hue='activity_level', ax = axes[1,
0],legend= False)
sns.scatterplot(data=df, x = 'calories', y ='very_active_minutes', hue='activity_level', ax = axes[1,1],l
egend= False)

handles, labels =axes[0,0].get_legend_handles_labels()

fig.legend(handles, labels,title = 'Activity_level',fontsize=16,title_fontsize=18, loc='center right', bb
ox_to_anchor=(1.9, 0.5))

plt.tight_layout(rect=[0,0,1,1])

fig.suptitle('Correlation Between Activity Level minutes and calories', x=0, y=1, fontsize=15)

plt.show()
```